Date Ierarhice in RDBMS

Introducere

- Exista momente cand avem nevoie sa stocam date arborescente sau ierarhice pentru a modela diverse probleme:
  - Categorii, sub-categorii si sub-sub-categorii intr-un online shop
  - Meniul de navigare intr-un sitemap complex
  - Relatiile dintre membrii unui sistem multilevel (eg: MLM)
- Organizarea datelor si indexarea ar trebui sa ofere:
  - Afisarea rezultatelor (cum se construieste un arbore de navigare?)
  - Raport statistic (cate persoane sunt pe o ramura?)
  - Extragera unui subarbore specific (se doreste afisarea produselor cu categorii dintr-o anumita sub-categorie, vezi cursul SD)

Arbori

Exista o diferenta intre arbori si ierarhii.

Arborii sunt un caz special de graf; ierarhiile sunt un caz special de arbori si au legatura cu mostenirea si subordonarea.

- Arborii sunt grafuri cu urmatoarele proprietati:

un arbore

  - este un graf conex fara cicluri. Un graf conex este un graf in care exista un drum intre oricare 2 noduri. Nu exista nod deconectat de restul grafului.
  - orice nod este radacina unui subarbore.
  - oricare 2 noduri sunt conectate de un singur drum.
  - Este un graf conex ce are numarul de muchii mai mic cu 1 decat numarul de noduri.

In arbore, cand se sterge o muchie, rezultatul este o padure (forest) cu 2 arbori disjuncti.

Ierarhii

- O ierarhie este un arbore directionat cu proprietati suplimentare de subordonare si mostenire. Relatia reprezentata de muchiile grafului poate fi de la radacina sau spre radacina:
  - Intr-o organizatie autoritatea porneste de obicei de la radacina, cu CEO, si descreste spre frunze.
- Mostenirea, fie de la sau spre radacina, este proprieatea cea mai importanta a unei ierarhii. Aceasta proprietate nu este caracteristica unui arbore.
  - O ierarhie permite unui nod sa aiba mai multe roluri. Intr-o organizatie o persoana poate avea mai multe roluri.
- Ierarhiile au roluri ce sunt ocupate de entitati. Proprietatea de rol nu exista intr-un arbore; fiecare nod apare o data intr-un arbore si este unic.

Tipuri de Ierarhii

Putem identifica 4 tipuri de modelare:

1. **Noduri statice si muchii statice**: schema contabila dintr-un sistem economic nu se va schima (mult/des) in timp.

2. **Noduri statice si muchii dinamice:** message board-ul unui forum/chat; continutul mesajelor (nodurile) nu se schimba dupa ce sunt postate; raspunsurile noi sunt postate ca subordonate ale unui mesaj existent (muchie noua).

3. **Noduri dinamice si muchii statice:** schema organizationala, in care organizarea de functii ramane aceeasi, oamenii ce ocupa aceste functii se schimba/muta.

4. **Noduri dinamice si muchii dinamice:** modelul graf al unei retele de transport sau comunicatii. Traficul in retea este in continua schimbare. Un exemplu: Waze, rute diferite la momente diferite.

Consideratii de Manipulare a Datelor

Problema poate fi privita din urmatoarele (nu e lista completa de) perspective relativ la manipularea datelor:

- inserarea unui nou nod
- stergerea unui nod
- mutarea unui nod
- navigarea in structura
  - gasirea descendentilor
  - gasirea ascendentilor
  - gasirea nivelului unui nod

Modele de Proiectare

In literatura se gasesc diverse modele cu caracteristici diferite pentru organizarea datelor ierarhice:

- liste de adiacenta (adjacency list)
- enumerarea cailor (path enumeration)
- multimi imbricate (nested sets)
- tabele de legatura (bridge table)
- intervale imbricate (nested intervals)
- tabele late (flat table)
- altele ...

Liste de Adiacenta

O metoda simpla (folosita anterior) de stocare si intretinere a ierarhiei sub forma de relatii parinte-fiu este de a inlude pentru fiecare inregistrare o referinta catre inregistrarea parinte. Acest model se numeste “Adjacency List Model”.

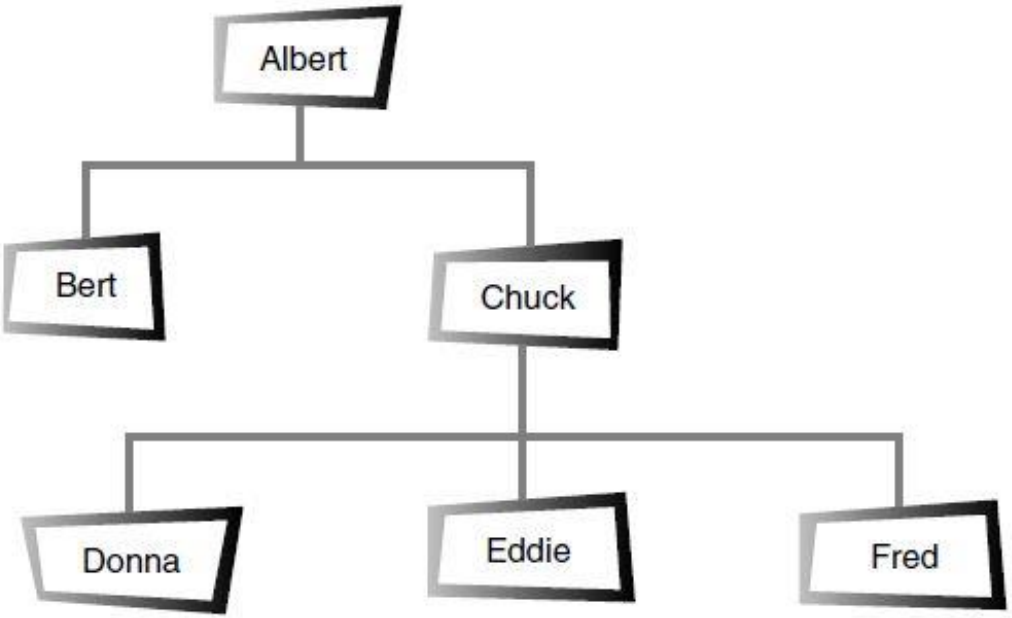
Acest model este un fel de graf de relatii si reprezinta un caz special in care graful formeaza o structura arborescenta.

Reprezentarea arborescenta permite interogarea usoara a nodurilor imediat adiacente unui nod dat (deasupra sau dedesubt in ierarhie)

Necesita recursivitate pentru interogarea datelor ce se gasesc pe mai multe nivele in arbore.

Liste de Adiacenta

```
CREATE TABLE Personnel_OrgChart
(emp VARCHAR(10) NOT NULL PRIMARY KEY,
boss VARCHAR(10), -- null means root
salary DECIMAL(6,2) NOT NULL,
... );
```



Liste de Adiacenta

- Cea mai des intalnita solutie
- Fiecare intrare isi cunoaste parintele
- Trebuiesc evitate unele anomalii:
  - anomalii de update (un angajat isi schimba numele)
  - anomalii de insert (crearea unui ciclu in graf prin inserarea unei relatii)
  - anomalii de delete (un angajat este sters si un subarbore devine deconectat)
  - anomalii de structura (referinta spre sine, null-uri multiple)
- Aceste anomalii pot aparea din cauza **DKNF**; este o forma normala folosita in normalizarea bazei de date ce necesita ca o tabela sa nu contina alte constrangeri decat de domeniu si cheie.

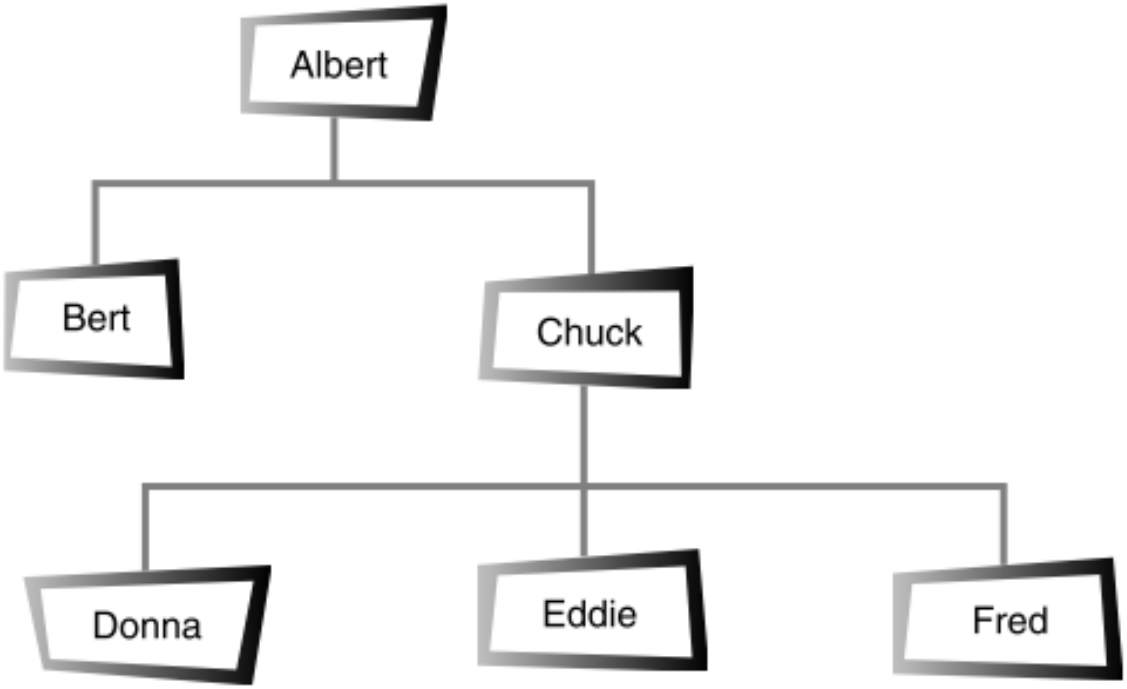
  Constangerea de domeniu specifica valorile permise pentru un atribut, iar cele de cheie specifica atributele ce identifica unic o inregistrare in tabela.

Liste de Adiacenta

```
CREATE TABLE Personnel
(emp_nbr INTEGER DEFAULT 0 NOT NULL PRIMARY
KEY,
emp_name VARCHAR(10) DEFAULT '{ {vacant} }'
NOT NULL,
address VARCHAR(35) NOT NULL,
birth_date DATE NOT NULL,
...);
```

Consideram ca exista un angajat virtual numit '{ {vacant} }' cu marca angajat (emp_nbr) egala cu valoarea 0

Liste de Adiacenta



Liste de Adiacenta

```
CREATE TABLE OrgChart
(job_title VARCHAR(30) NOT NULL PRIMARY KEY,
emp_nbr INTEGER DEFAULT 0 NOT NULL REFERENCES
Personnel(emp_nbr)
ON DELETE SET DEFAULT ON UPDATE CASCADE,
-- zero is vacant position
boss_emp_nbr INTEGER -- null means root node
REFERENCES Personnel(emp_nbr),
UNIQUE (emp_nbr, boss_emp_nbr),
salary DECIMAL (12,4) NOT NULL CHECK (salary
>= 0.00),
...);
```

Liste de Adiacenta

Trebuie sa adaugam constrangeri tabelei pentru a forta proprietatea de arbore (de asemenea sa asiguram faptul ca functia nu e detinuta de o persoana ce nu este angajata in companie)

Conditia ar putea fi:

```
CHECK (boss_emp_nbr <> emp_nbr) -- cannot be
your own boss!
```

dar, nu e suficient din cauza angajatului virtual cu marca 0, functia 'vacant'.

```
CHECK ((boss_emp_nbr <> emp_nbr) OR
(boss_emp_nbr = 0 AND emp_nbr = 0))
```

Alte cicluri sunt evitate folosind constrangerea `UNIQUE (emp, boss)` ce limiteaza la 1 singur manager (boss) intre angajati.

Liste de Adiacenta

Din teoria grafelor stim ca numarul de muchii intr-un arbore este numarul de noduri - 1:

```
CHECK ((SELECT COUNT(*) FROM OrgChart) - 1
-- edges
= (SELECT COUNT(boss_emp_nbr) FROM OrgChart))
-- nodes
```

Conditia `COUNT (boss_emp_nbr)` va elimina NULL din radacina.

Are acelasi efect cu a avea o constrangere de verificare a unui NULL:

```
CHECK((SELECT COUNT(*) FROM OrgChart WHERE
boss_emp_nbr IS NULL) = 1)
```

Este o conditie necesara, dar nu si suficienta.

Ex: emp1 este boss pentru emp2 si emp2 este boss pentru emp1

Liste de Adiacenta

O abordare posibila ar fi eliminarea tuturor nodurilor frunza si repetarea procedurii pana cand se obtine multimea vida. Daca arborele nu se reduce la multimea vida inseamna ca exista un ciclu deconectat.

```
CREATE FUNCTION TreeTest() RETURNS CHAR(6)
LANGUAGE SQL
DETERMINISTIC
BEGIN ATOMIC
-- put a copy in a temporary table
INSERT INTO Tree SELECT emp, boss FROM
Personnel_OrgChart;
-- prune the leaves
WHILE (SELECT COUNT(*) FROM Tree) - 1
= (SELECT COUNT(boss) FROM Tree)
```

Liste de Adiacenta

```
DO DELETE FROM Tree
WHERE Tree.emp
NOT IN (SELECT T2.boss
FROM Tree AS T2
WHERE T2.boss IS NOT NULL);
IF NOT EXISTS (SELECT * FROM Tree)
THEN RETURN ('Tree');
ELSE RETURN ('Cycles');
END IF;
END WHILE;
END;
```

Liste de Adiacenta

Structura originala Personnel_OrgChart se poate reconstitui usor cu urmatorul VIEW:

```
CREATE VIEW Personnel_OrgChart (emp_nbr, emp,
boss_emp_nbr, boss)
AS SELECT E1.emp_nbr, E1.emp_name,
E1.boss_emp_nbr, B1.emp_name
FROM Personnel AS E1, Personnel AS B1,
OrgChart AS O1
WHERE B1.emp_nbr = O1.boss_emp_nbr
AND E1.emp_nbr = O1.emp_nbr;
```

NULL in Liste de Adiacenta

Utilizarea NULL pentru a reprezenta radacina nu este singura modalitate;

Sa ne aducem aminte cum am rezolvat la cursul SD in semestrul anterior

Navigarea in Liste de Adiacenta

- foarte simplu de obtinut **copiii unui nod**

```
SELECT * FROM OrgChart AS O1 LEFT JOIN
OrgChart AS O2 ON O2.emp_nbr = O1.boss_emp_nbr
```

- foarte simplu de obtinut **parintele unui nod**

```
SELECT * FROM OrgChart AS O1 INNER JOIN
OrgChart AS O2 ON O1.emp_nbr = O2.boss_emp_nbr
```

- Mai dificil de afisat **subarborele complet al unui nod**

```
SELECT * FROM OrgChart AS O1 LEFT JOIN
OrgChart AS O2 ON O2.emp_nbr = O1.boss_emp_nbr
LEFT JOIN OrgChart AS O3 ON O3.emp_nbr =
O2.boss_emp_nbr
```

…

- SQL-99 introduce recursive CTE

```
WITH [RECURSIVE] ...
```

Navigarea in Liste de Adiacenta

- **Inserarea unui nod**: doar se insereaza perechea (emp_nbr, boss_emp_nbr) in tabel; nu trebuie schimbat niciun alt rand.
- S**tergerea unui nod**:
  - Stergerea unui nod frunza: doar se sterge randul din structura arborescenta.
  - Cand se sterge un nod intern din arbore problema este complexa. Este necesara o decizie asupra nodurilor subordonate; exista mai multe abordari:

1. Ancient Egyptian school of management: cand se sterge un nod, toate nodurile subordonate se sterg. (Cand moare faraonul, se ingroapa toti sclavii cu el.)

2. Send the orphans to Grandmother: nodurile subordinate nodului sters devin noduri subordonate ale parintelui nodului sters (bunic).

Navigarea in Liste de Adiacenta

3. The oldest son takes over the shop: unul din nodurile subordonate preia locul nodului sters. Aceasta promovare poate provoca o cascada de promovari in subarbore

Enumerarea Cailor

Se bazeaza pe proprietatea arborilor ca exista doar cate o singura cale de la radacina la orice nod din arbore. Modelul stocheaza calea sub forma de string prin concatenarea fie a muchiilor fie a cheilor nodurilor din cale. Cautarile se fac pe baza aplicarii de functii pentru stringuri.

Exista 2 metode pentru enumerarea cailor: **enumerarea muchiilor** si **enumerarea nodurilor**. Enumerarea nodurilor este metoda mai des folosita; de altfel exista o mica diferenta in operatiile pe stringuri necesare pentru cele 2 modele.

```
--CHECK() prevents separator in the column.
CREATE TABLE Personnel_OrgChart
(emp_name CHAR(10) NOT NULL,
emp_id CHAR(1) NOT NULL PRIMARY KEY
CHECK(REPLACE (emp_id, '/', '') = emp_id) ),
path_string VARCHAR(500) NOT NULL);
```

Enumerarea Cailor

Este necesara prevenirea ciclurilor in graf: un ciclu s-ar reprezenta ca un string de cale in care cel putin o cheie (emp_id) apare de 2 ori. Acest lucru se poate obtine cu o constrangere ce foloseste o subinterogare:

```
CHECK (NOT EXISTS
(SELECT *
FROM Personnel_OrgChart AS D1,
Personnel_OrgChart AS P1
WHERE CHAR_LENGTH (REPLACE (D1.emp_id,
P1.path_string, ‘’))
< (CHAR_LENGTH(P1.path_string)- 1) --size of one
emp_id string
) )
```

Enumerarea Cailor

Nicio cale nu poate avea lungimea mai mare decat numarul de noduri din arbore

```
CHECK ((SELECT MAX(CHAR_LENGTH(path_string) )
FROM Personnel_OrgChart AS P1)
<= (SELECT COUNT(emp_id) * CHAR_LENGTH(emp_id)
FROM Personnel_OrgChart AS P2))
```

Se presupune ca cheia (emp_id) are lungime fixa si nu se foloseste niciun separator in stringul de cale.

Enumerarea Cailor

emp_name	emp_id	path_string
'Albert'	'A'	'A'
'Bert'	'B'	'AB'
'Chuck'	'C'	'AC'
'Donna'	'D'	'ACD'
'Eddie'	'E'	'ACE'
'Fred'	'F'	'ACF'

Enumerarea Cailor

Pentru un anumit nod, **gasiti toti subarborii sai**. Cea mai simpla solutie:

```
SELECT *
FROM Personnel_OrgChart
WHERE path_string LIKE '%' || :parent_emp_id ||
'%';
```

Dezavantaj:

Cautarile folosind predicate LIKE ce incep cu wildcard '%' nu pot folosi indexuri si gereaza table scan.

Enumerarea Cailor

O alta posibila abordare este utilizarea interogarii:

```
SELECT * FROM Personnel_OrgChart
WHERE path_string
LIKE (SELECT path_string
FROM Personnel_OrgChart
WHERE emp_id = :parent_emp_id) || '%';
```

Subinterogarea va putea folosi un index pe coloana emp_id pentru a gasi “inceputul” stringului, de la radacina la parintele nodului de interes.

Enumerarea Cailor

Navigarea in adancime in arbore e usoara. In loc de wildcard-ul ‘%’, se foloseste un string ce contine wildcard-uri underscore (‘_’) de lungime potrivita. De exemplu, interogarea de mai jos va returna copiii directi ai unui nod parinte dat (emp_id):

```
SELECT * FROM Personnel_OrgChart
WHERE path_string
LIKE (SELECT path_string
FROM Personnel_OrgChart
WHERE emp_id = :parent_emp_id) ||'_';
```

Enumerarea Cailor

Navigarea in adancime in arbore e usoara. In loc de wildcard-ul ‘%’, se foloseste un string ce contine wildcard-uri underscore (‘_’) de lungime potrivita. De exemplu, interogarea de mai jos va returna copiii directi ai unui nod parinte dat (emp_id):

```
SELECT * FROM Personnel_OrgChart
WHERE path_string LIKE (SELECT path_string
FROM Personnel_OrgChart
WHERE emp_id = :parent_emp_id) || REPLICATE(‘_’,
:n);
```

Enumerarea Cailor

Pentru un nod dat, **sa se gaseasca toti superiorii** (ascendentii). Aceasta problema necesita dezasamblarea caii in identificatorii componenti. Se poate utiliza o tabela de secvente de intregi pentru a determina substringul cerut:

```
SELECT SUBSTRING (P1.path_string
FROM (seq * CHAR_LENGTH(P1.emp_id) )
FOR CHAR_LENGTH(P1.emp_id) ) AS emp_id
FROM Personnel_OrgChart AS P1, Sequence AS S1
WHERE P1.emp_id = :search_emp_id
AND S1.seq <=
CHAR_LENGTH(path_string)/CHAR_LENGTH(emp_id);
```

Enumerarea Cailor

Problema este ca nu vom obtine si relatia intre superiori, doar lista acestora.

```
SELECT P2.*
FROM Personnel_OrgChart AS P1,
Personnel_OrgChart AS P2
WHERE P1.emp_id = :search_emp_id
AND POSITION (P2.path_string IN P1.path_string) =
1;
```

Enumerarea Cailor

Stergerea subarborelui cu radacina in acel nod se poate face cu acelasi predicat utilizat la gasirea subordonatilor:

```
DELETE FROM Personnel_OrgChart
WHERE path_string
LIKE (SELECT path_string
FROM Personnel_OrgChart
WHERE emp_id = :dead_guy) || '%';
```

Enumerarea Cailor

Acest model are aceleasi proprietati de **inserare** cu modelul lista de adiacenta. Noul nod (emp_id) se concateneaza la sfarsitul caii nodului parinte.

```
INSERT INTO Personnel_OrgChart VALUES (:new_emp,
:new_emp_id,
(SELECT path_string FROM Personnel_OrgChart WHERE
emp_id= :new_emp_boss) || :new_emp_id);
```

inserarea unui subarbore:

```
INSERT INTO Personnel_OrgChart SELECT N1.emp,
N1.emp_id,
(SELECT path_string FROM Personnel_OrgChart WHERE
emp_id= :new_tree_boss) || N1.emp_id
FROM NewTree AS N1;
```

Multimi Imbricate

Foloseste o tehnica ce utilizeaza 2 valori aditionale pentru fiecare nod, conform cu traversarea arborelui, ce viziteaza fiecare nod de 2 ori, atribuind valorile (numere) in ordinea vizitarii, la ambele vizite.

Cele 2 valori numerice sunt stocate ca 2 atribute. Apartenenta la ierarhie este simplu de verificat prin compararea celor 2 valori.

Actualizarea este mult mai costisitoare si necesita renumerotare. Se pot face anumite imbunatatiri folosind numere rationale in loc de intregi, in felul acesta evitand renumerotarea, obtinand o actualizare mai rapida, dar mai complexa.

Multimi Imbricate

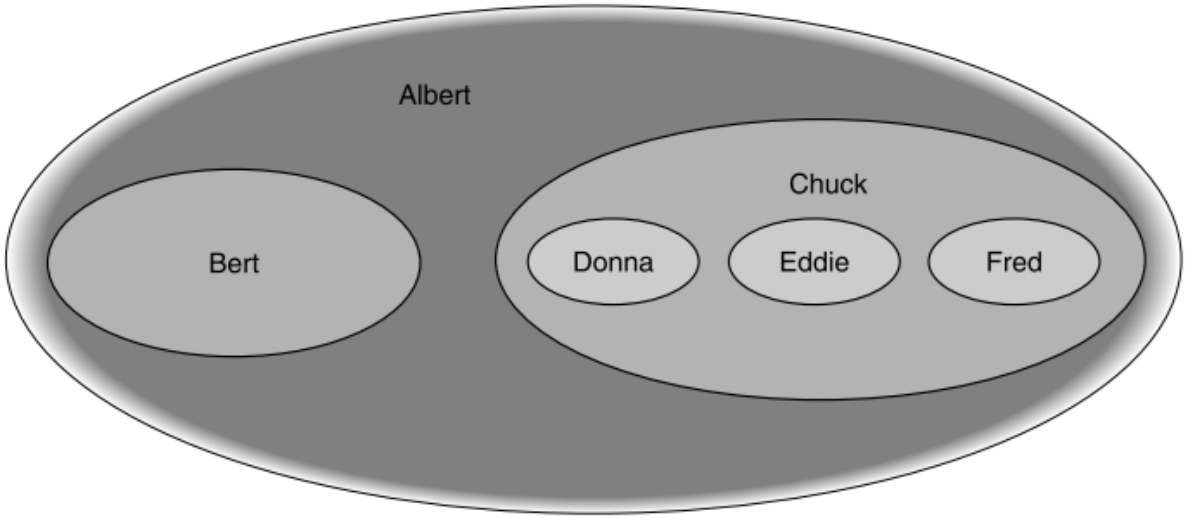
Pentru reprezentarea grafica a unui arbore sub forma de multimi imbricate se pot folosi elipse si imbricarea elipselor subordonate in interiorul parintilor. Daca o elipsa este continuta intr-o alta, va reprezenta subordonare.

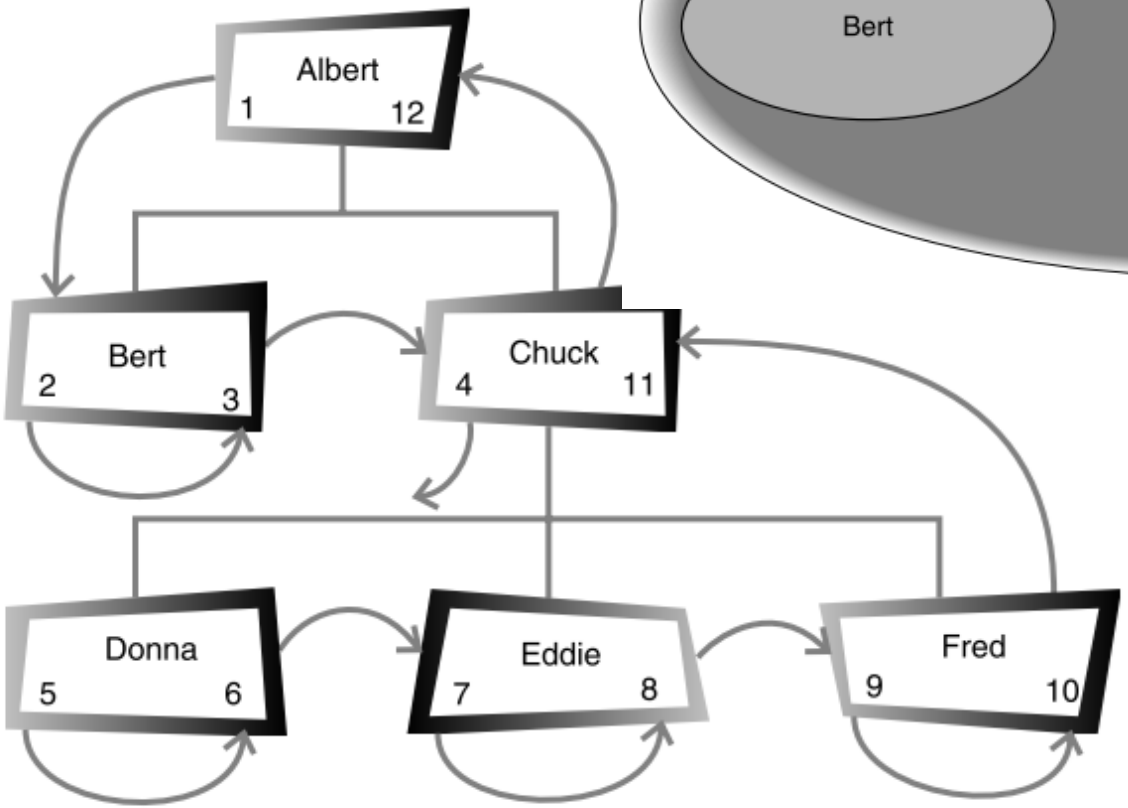
Radacina va fi reprezentata de cea mai mare (exterioara) elipsa si va contine toate celelalte noduri. Nodurile frunza vor fi reprezentate de elipsele cele mai interioare, fara a contine alte elipse, iar imbricarea elipselor va arata relatia ierarhica.

Putem modela arborele cu (lft, rgt) multimi imbricate folosind perechi de intregi.

Aceste perechi de intregi vor contine intotdeauna perechile corespunzand nodurilor subordonate, astfel ca un nod copil va fi intre limitele nodului parinte.

Multimi Imbricate





Multimi Imbricate

Nodul radacina va avea intotdeauna ca in coloana lft valoarea 1, iar in cloana rgt valoarea dublului numarului de noduri.

Radacina se poate determina cu interogarea:

```
SELECT * FROM Orgchart WHERE lft = 1;
```

Interogarea poate beneficia de existenta unui index pe valorile coloanei lft. Un nod frunza este unul ce nu are niciun copil. In modelul liste de adiacenta nu se determina foarte usor astfel de noduri:

```
SELECT * FROM OrgChart AS O1
WHERE NOT EXISTS
(SELECT * FROM OrgChart AS O2
WHERE O1.member = O2.boss);
```

Multimi Imbricate

In modelul de multimi imbricate pentru nodurile frunza avem proprietatea ca diferenta intre valoarea lui lft si cea a lui rgt este intotdeauna 1. Putem determina nodurile frunza:

```
SELECT * FROM Orgchart WHERE(rgt - lft) = 1;
```

Pentru a putea eficientiza astfel de determinari se poate utiliza un index fie pe coloana lft, fie pe perechea de coloane (lft, rgt), optimizatorul putand utiliza acel index in loc de un table scan:

```
SELECT * FROM Orgchart
WHERE lft =(rgt - 1);
```

Multimi Imbricate

Un arbore este un graf fara cicluri; de asemenea, exista intotdeauna o cale de la radacina la orice nod din arbore.

O alta proprietate este ca orice nod in arbore este radacina unui subarbore; unele proprietati ale acelui subarbore sunt utile perechii de coloane (lft, rgt). In tabela multimi imbricate toti descendentii unui nod pot fi regasiti prin cautarea nodurilor ce au valorile (lft, rgt) intre valorile (lft, rgt) ale nodului parinte.

Toti subordonatii fiecarui boss din ierarhia organizationala:

```
SELECT Mgrs.member AS boss, Workers.member AS
worker
FROM Orgchart AS Mgrs, Orgchart AS Workers
WHERE Workers.lft BETWEEN Mgrs.lft AND Mgrs.rgt
AND Workers.rgt BETWEEN Mgrs.lft AND Mgrs.rgt;
```

Multimi Imbricate

Nivelul unui nod in arbore este numarul de muchii dintre acel nod si radacina. O cale este o multime de muchii ce conecteaza 2 noduri. Modelul multimi imbricate se foloseste de faptul ca fiecare pentru fiecare parinte are “latimea” (latimea = [rgt - lft]) mai mare decat a multimilor continute. Radacina va avea “latimea’ cea mai mare.

Pentru a determina nivelul fiecarui nod (employee) :

```
SELECT O2.member, COUNT(O1.member) AS level
FROM OrgChart AS O1, OrgChart AS O2
WHERE O2.lft BETWEEN O1.lft AND O1.rgt
GROUP BY O2.member;
```

Expresia COUNT(01.member) va contoriza si nodul insusi.

Multimi Imbricate

Pentru a **sterge un nod intern** din arbore este mai dificil decat eliminarea intregului subarbore in modelul nested sets. Cand se elimina nodul intern trebuie decis cum se va “umple” nodul lipsa. Se pot aplica mai multi algoritmi. O metoda este de a conecta copii la parintele nodului de sters (Mom dies and Grandma adopts the kids). Reprezinta o promovare verticala a intregului subarbore.

O alta promovare verticala este de a muta un singur copil pe pozitia nodului sters (give the business to the oldest son). Dezavantajul este ca atunci cand nodul copil este promovat va determina o lipsa in acea pozitie.

O alta solutie este promovarea orizontala. Nodul frate din dreapta al nodului sters se muta pe pozitia vacanta (Dad dies and his oldest brother takes over the business). Se presupune ca exista un astfel de nod frate care sa preia acea pozitie.

Conversia Nested Sets la Adjacency List

Modelul adjacency list este cel mai des utilizat pentru ca:

- E. F. Codd a publicat o lucrare ce utiliza adjacency list, el fiind parintele domeniului BD.
- Reprezinta o modalitate de a construi inlantuiri de pointeri, metoda traditionala de programare pentru gestiunea arborilor in limbajele procedurale.

```
SELECT B.member AS boss, P.member
FROM OrgChart AS P LEFT OUTER JOIN
Personnel AS B
ON B.lft = (SELECT MAX(S.lft)
FROM OrgChart AS S
WHERE P.lft > S.lft AND P.lft < S.rgt);
```

Conversia Nested Sets la Adjacency List

O alta versiune a aceleiasi interogari:

```
SELECT B.member AS boss, P.member
FROM OrgChart AS B, Personnel AS P
WHERE P.lft BETWEEN B.lft AND B.rgt
AND B.member = (SELECT MAX(S.member)
FROM OrgChart AS S
WHERE S.lft < P.lft AND S.rgt > P.rgt);
```

Versiunea aceasta ar trebui sa fie mai simpla si mai rapida. Rezultatul va contine doar muchiile grafului, nu si o inregistrare (NULL, <root>).

Tabele de Legatura

Modalitatea cea mai intalnita pentru stocarea ierarhii este sa utilizam un "pointer" (foreign key) spre parinte. Una din probleme este ca in unele versiuni de SGBD nu se poate obtine simplu un intreg arbore. Problema este asemanatoare cu regasirea tuturor descendentilor sau ascendentilor unui nod.

Modelele prezentate incerca sa depaseasca aceste probleme dar au fiecare dezavantaje. Problema este ca gestiunea trebuie facuta la nivel de aplicatie: trebuie aleasa performanta fie pentru timpul de regasire a datelor, fie pentru cel de insert/delete.

Mai exista un model numit **tabel de legatura (bridge table/closure table).** Modelul permite regasirea tuturor descendentilor unui nod (**optional pana la o adancime specificata**) doar cu un join cu tabela de legatura. Avantajul principal este ca atunci cand se adauga sau sterge o relatie parinte-copil, doar o operatie este necesara (poate fi configurata sa ruleze ca trigger pe tabela principala).

Tabele de Legatura

O tabela de legatura (closure table) este o tabela ce stocheaza inchiderea tranzitiva ("transitive closure“) a relatiilor parinte-copil din tabela de baza.

In aceasta structura, avand o tabela cu campuri pentru parinte, copil si adancime, ierarhia poate fi gestionata pentru orice adancime.

Numarul total de inregistrari in tabela de legatura este egal cu numarul de inregistrari din tabela de baza inmultit cu adancimea medie a arborelui din tabela de baza. Fiecare inregistrare din tabela de baza are o inregistrare corespondenta in tabela de legatura, plus o inregistrare pentru fiecare parinte.

Tabele de Legatura

Pentru a face nodul PARENT_ITEM parinte al nodului CHILD_ITEM, codul poate fi:

```
insert into closure(parent, child, depth)
select p.parent, c.child, p.depth+c.depth+1
  from closure p, closure c
 where p.child=PARENT_ITEM and
c.parent=CHILD_ITEM
```

Costul acestei operatii este direct proportional cu marimea subarborelui cu radacina CHILD_ITEM; daca se adauga o frunza, este doar un simplu insert de inregistrari pentru parinte pentru noul nod. Daca se adauga un parinte la un arbore existent, se va insera o noua inregistrare pentru fiecare nod “adoptat” din subarbore.

Tabele de Legatura

Elementul principal pentru tabela de legatura sunt inregistrarile cu depth 0. Fara ele, 4 inserari separate ar fi necesare, sau o interogare cu UNION pentru a colecta relatiile unui nou link.

Stergerea unei legaturi (link) poate utiliza o interogare precum:

```
delete link
  from closure p, closure link, closure c
 where p.parent = link.parent and c.child =
link.child and p.child=PARENT_ITEM    and
c.parent=CHILD_ITEM
```

Tabele de Legatura

Datorita faptului ca inserarea si stergerea sunt simple si modifica doar tabela de legatura, se pot automatiza aceste operatii cu triggere pe tabela de baza.

Sunt necesare actiuni suplimentare: in momentul inserarii, se insereaza inregistrarea (self,self,0) in tabela de legatura, iar in momentul stergerii, se sterge atat self-link-ul cat si toate legatura care depind de el:

```
delete link
  from closure p, closure link, closure c,
closure to_delete
 where p.parent=link.parent and
c.child=link.child and p.child=to_delete.parent
and c.parent= to_delete.child and
(to_delete.parent=DELETE_ITEM or
to_delete.child=DELETE_ITEM) and
to_delete.depth<2
```

Intervale Imbricate

Reprezinta generalizarea Multimilor Imbricate folosind numerele rationale (de exemplu numere rationale diadice)

Multimile Imbricate sunt potrivite pentru unele aplicatii, dar au si dezavantaje:

1. Codificare este volatila. In jur de jumatate din nodurile arborelui ar trebui reetichetate cand se adauga un nod.

2. Interogarea intervalelor este asimetrica din perspectiva performantei. Este usor de raspuns daca un punct este in interval, dar este greu de indexat o multime de intervale ce contin un anumit punct. Acest fapt conduce la dificultatea de a raspunde la interogari despre ascendentii unui nod.

Intervale Imbricate

Modelul Nested Sets (Seturi Imbricate) folosește numere întregi pentru a codifica nodurile, ceea ce înseamnă că există spații finite (lacune) pentru inserarea de noi noduri. Pentru a depăși această problemă, se poate utiliza un domeniu mai dens, cum ar fi numerele raționale.

Modelul Nested Intervals (Intervale Imbricate) codifică fiecare nod al arborelui cu o pereche de numere, numită cap (head) și coadă (tail). Intervalul alocat unui nod copil este întotdeauna conținut în intervalul nodului părinte.

Cu aceasta etichetare inchiderea tranzitiva poate fi interogata astfel:

```
select e1.ename, e2.ename
from emp e1, emp e2
where e2.head >= e1.head
and e2.head < e1.tail
```

Intervale Imbricate

Subarborele tuturor descendentilor unui nod poate fi regasit prin restrictia asupra view-ului anterior cu un predicat

```
select e2.ename from emp e1, emp e2
where e2.head >= e1.head
and e2.head < e1.tail and e1.ename = 'SCOTT'
```

Calea pentru ascendeti poate fi obtinuta simetric

```
select e1.ename from emp e1, emp e2
where e2.head >= e1.head
and e2.head < e1.tail and e2.ename = 'SCOTT'
```

Tabela Flat

- O Lista de Adiacenta modificata prin adaugarea coloanelor Level si Rank (ordonare) pentru fiecare inregistrare.
- Simplu de iterat/paginat
- Sterge si mutare costisitoare
- Utile la: discutii/formuri/comentarii pe bloguri, organizate in threaduri.

Implementari

http://www.mysqltutorial.org/mysql-adjacency-list-tree/

https://docs.microsoft.com/en-us/sql/relational-databases/hierarchical-data-sql-server?view=sql-server-ver15

https://coderwall.com/p/whf3-a/hierarchical-data-in-postgres

http://www.klempert.de/php/nested_sets/demo/ (germana)

Resurse

- https://www.databasestar.com/hierarchical-data-sql/
- https://learnsql.com/blog/how-to-query-hierarchical-data/